

Linux - Master Notes

A Complete Reference for Interview Preparation, Production Work, Real-World Projects, and System Understanding.

Java Full Stack Master Course

Table of Contents

LINUX — MASTER NOTES	5
A Complete Reference Book for Interviews, Production Work & System Understanding	5
TOPIC 1: FILE SYSTEM	5
1. Introduction	5
1.1 Why Linux, and why does this matter to you specifically?	5
1.2 The Linux Filesystem Hierarchy	5
2. Real Life Analogy	5
3. Frequently Asked Interview Questions	5
TOPIC 2: COMMANDS	6
1. Introduction	6
2. Essential Commands — Full Reference	6
3. Piping and Redirection — Combining Commands	6
4. Real Life Analogy	6
5. Frequently Asked Interview Questions	6
TOPIC 3: USERS & GROUPS	7
1. Introduction	7
2. Key Commands	7
3. sudo — Temporarily Elevated Privileges	7
4. Real Life Analogy	7
5. Frequently Asked Interview Questions	7
TOPIC 4: PERMISSIONS	8
1. Introduction	8
2. Reading Permission Output	8
3. Changing Permissions — chmod	8
The Numeric System (Frequently Tested)	8
4. Changing Ownership — chown	8

5. Real Life Analogy	8
6. Edge Cases	9
7. Common Mistakes	9
8. Frequently Asked Interview Questions	9

TOPIC 5: SSH **9**

1. Introduction	9
2. Recap and Deeper Detail — Key-Based Authentication	9
3. Generating Your Own SSH Key Pair	10
4. Copying Files Over SSH — scp (Recap From AWS)	10
5. Real Life Analogy	10
6. Frequently Asked Interview Questions	10

TOPIC 6: PROCESSES **10**

1. Introduction	10
2. Key Commands	10
3. Foreground vs Background Processes	11
4. Real Life Analogy	11
5. Frequently Asked Interview Questions	11

TOPIC 7: CRON JOBS **11**

1. Introduction	11
2. Code Example — Editing the Crontab	11
3. Cron vs Spring's @Scheduled — When to Use Which	12
4. Real Life Analogy	12
5. Frequently Asked Interview Questions	12

TOPIC 8: SHELL SCRIPTING — BASICS **12**

1. Introduction	12
2. Code Example — A Deployment Script	13
3. Real Life Analogy	13
4. Frequently Asked Interview Questions	13

TOPIC 9: NGINX	13
1. Introduction	14
2. Code Example — A Basic Reverse Proxy Configuration	14
3. Why Put Nginx IN FRONT of Spring Boot's Embedded Tomcat?	14
4. Real Life Analogy	14
5. Frequently Asked Interview Questions	14
 TOPIC 10: LOGS	 15
1. Introduction	15
2. Common Log Locations	15
3. Practical Log Investigation (Recapping Topic 2's Commands, Applied Together)	15
4. Real Life Analogy	15
5. Frequently Asked Interview Questions	15
6. Revision Notes (Entire Linux Section Consolidated)	16
7. Homework (Covers Entire Linux Section)	16
FINAL SUMMARY — LINUX COMPLETE	16

LINUX — MASTER NOTES

A Complete Reference Book for Interviews, Production Work & System Understanding

TOPIC 1: FILE SYSTEM

1. Introduction

1.1 Why Linux, and why does this matter to you specifically?

Recall from the AWS section: your EC2 instance ran **Amazon Linux**. Recall from the Docker section: EVERY Docker image is built on a Linux base (eclipse-temurin:17-jre-alpine is Linux underneath). Recall from CI/CD: GitHub Actions' ubuntu-latest runners are Linux. **Virtually the ENTIRE deployment target for everything you've built in this course runs on Linux** — understanding it is not optional trivia, it's where your Java application ACTUALLY lives in production.

1.2 The Linux Filesystem Hierarchy

```
/ <- ROOT - everything starts here (unlike Windows' separate C:\, D:\ drives)
■■■ bin/ <- essential command binaries (ls, cp, mv)
■■■ etc/ <- system CONFIGURATION files (nginx.conf, systemd services)
■■■ home/ <- user home directories (/home/ec2-user, matching your AWS EC2 login!)
■■■ var/ <- VARIABLE data - logs (/var/log), that changes often
■■■ tmp/ <- temporary files, often cleared on reboot
■■■ usr/ <- installed user programs/libraries
■■■ opt/ <- optional/third-party application installations (often where you'd put your .jar)
```

2. Real Life Analogy

- **Bank:** The Linux filesystem is like a bank's ENTIRE building having ONE single, unified addressing system starting from the main entrance (/) — every single room, from the vault (/etc — critical configuration) to the break room (/tmp — temporary, disposable) has a clear, predictable place within that ONE unified structure, unlike having several completely separate buildings (Windows' multiple drive letters).

3. Frequently Asked Interview Questions

- 1 **What is the root of the Linux filesystem?** / — everything in Linux starts from this single root, unlike Windows' separate drive letters.
- 2 **Where do system configuration files typically live?** /etc.
- 3 **Where do log files typically live?** /var/log.
- 4 **Why does understanding Linux matter for a Java developer specifically?** Because EC2 instances, Docker base images, and CI/CD runners throughout this entire course all run on Linux — it's where your Spring Boot application actually executes in production.

TOPIC 2: COMMANDS

1. Introduction

What is it? The core, everyday commands used to navigate and manipulate the Linux filesystem/system directly from a terminal — the SAME terminal you used to SSH into your EC2 instance (AWS section) and deploy your Spring Boot jar.

2. Essential Commands — Full Reference

```
pwd # print WORKING DIRECTORY (where am I right now?)
ls -la # LIST files, '-l' = long format (permissions, owner, size), '-a' = show hidden files
cd /home/ec2-user # CHANGE DIRECTORY
mkdir my-app # MAKE a new DIRECTORY
touch app.log # create an EMPTY file (or update its timestamp if it exists)
cp app.jar backup.jar # COPY a file
mv app.jar /opt/myapp/ # MOVE (or rename) a file
rm old-file.txt # REMOVE (delete) a file
rm -rf old-directory/ # remove RECURSIVELY and FORCEFULLY - DANGEROUS, no undo!
cat application.properties # print a file's ENTIRE content to the screen
less app.log # view a LARGE file page-by-page (better than 'cat' for big files)
tail -f app.log # follow a file's LIVE tail - watch new log lines APPEAR in real time
grep "ERROR" app.log # SEARCH for lines containing "ERROR"
find / -name "*.jar" # FIND files by name, searching recursively
chmod +x deploy.sh # make a script EXECUTABLE (Topic 4!)
df -h # DISK FREE - how much storage is used/available
```

3. Piping and Redirection — Combining Commands

```
grep "ERROR" app.log | wc -l # PIPE ('|') - feed one command's OUTPUT as the NEXT command's INPUT
# (count HOW MANY error lines exist)
java -jar app.jar > output.log 2>&1 # REDIRECT stdout (>) AND stderr (2>&1) into a file, instead of
the console
```

4. Real Life Analogy

- **Restaurant:** Commands are like a chef's standard hand tools — `ls` is like glancing at what's currently on the counter, `grep` is like quickly SEARCHING through a stack of order tickets for a specific dish name, and PIPING (`|`) is like passing a partially-prepared ingredient DIRECTLY to the next station instead of setting it down and having someone else pick it back up separately.

5. Frequently Asked Interview Questions

- 1 **What does `tail -f` do, and when is it commonly used?** Continuously displays new lines APPENDED to a file in real time — commonly used to watch a live application log (like your Spring Boot app's output) as it runs.
- 2 **What does the pipe (`|`) operator do?** Feeds one command's output directly as the input to the NEXT command, allowing commands to be chained together.

- 3 **Why is `rm -rf` considered dangerous?** It recursively and forcefully deletes files/directories with NO confirmation and NO undo — a single mistyped path can cause catastrophic, unrecoverable data loss.
-

TOPIC 3: USERS & GROUPS

1. Introduction

What is it? Linux is a MULTI-USER operating system by design — every process/file is OWNED by a specific USER, and users can belong to GROUPS — directly analogous to your IAM Users/Groups from the AWS section, just at the OPERATING SYSTEM level instead of the cloud platform level.

2. Key Commands

```
whoami # who am I currently logged in as?
sudo useradd deploy-user # CREATE a new user (recall the AWS section's 'ec2-user' - already created for you)
sudo passwd deploy-user # set that user's password
groups ec2-user # list which GROUPS a user belongs to
sudo usermod -aG docker ec2-user # ADD ec2-user to the 'docker' GROUP (a common REAL requirement - # letting a non-root user run docker commands without 'sudo' every time)
```

3. `sudo` — Temporarily Elevated Privileges

```
sudo systemctl restart nginx # 'sudo' runs THIS ONE command with ROOT (administrator) privileges
```

This is directly analogous to the "least privilege" IAM principle (AWS section) — you operate as a REGULAR, limited user day-to-day, and only ELEVATE to root privileges for the SPECIFIC commands that genuinely require it, rather than staying logged in as root constantly (mirroring the EXACT "never use root day-to-day" AWS guidance).

4. Real Life Analogy

- **Bank:** Linux users/groups are like the bank's staff ID badge system (directly recapping IAM from the AWS section) — each staff member (user) has their OWN identity and belongs to specific role GROUPS ("Tellers," "Managers"), and `sudo` is like a teller temporarily using a SUPERVISOR override key for ONE specific sensitive action, rather than being permanently issued supervisor-level access for their entire shift.

5. Frequently Asked Interview Questions

- 1 **Why is Linux described as "multi-user"?** Every process and file is owned by a specific user, and multiple distinct users can operate on the same system simultaneously, each with their own permissions.
- 2 **What does `sudo` do?** Temporarily elevates a command to run with root (administrator) privileges, rather than requiring the user to be permanently logged in as root.

- 3 **Why would you add a user to the `docker` group specifically?** To allow that user to run `docker` commands directly without needing `sudo` for every single one — a common real-world convenience/security trade-off.

TOPIC 4: PERMISSIONS

1. Introduction

What is it? Every file/directory in Linux has permissions controlling WHO can Read, Write, or Execute it — for THREE separate categories: the OWNER, the OWNING GROUP, and EVERYONE ELSE.

2. Reading Permission Output

```
ls -l app.jar
-rwxr-xr-- 1 ec2-user developers 15234 Jan 15 10:00 app.jar

-rwxr-xr--
| | | |
| | | +-- OTHERS: r-- (read only)
| | +----- GROUP: r-x (read + execute)
| +----- OWNER: rwx (read + write + execute)
+----- file TYPE (- = regular file, d = directory)
```

3. Changing Permissions — `chmod`

```
chmod +x deploy.sh # add EXECUTE permission (recap from Topic 2!)
chmod 755 deploy.sh # NUMERIC form: owner=7(rwx), group=5(r-x), others=5(r-x)
chmod 644 application.properties # owner=6(rw-), group=4(r--), others=4(r--) - standard for a config file
```

The Numeric System (Frequently Tested)

```
r (read) = 4 w (write) = 2 x (execute) = 1

7 = rwx (4+2+1) 6 = rw- (4+2) 5 = r-x (4+1) 4 = r-- (4)
```

4. Changing Ownership — `chown`

```
sudo chown ec2-user:developers app.jar # change the OWNER to 'ec2-user' and the GROUP to 'developers'
```

5. Real Life Analogy

- **Bank:** File permissions are like a bank's document classification stamps — "OWNER can fully edit," "the RELEVANT DEPARTMENT (group) can read and process," "GENERAL STAFF (others) can only view" — a SEPARATE, clearly labeled access level for each of the three distinct audiences, applied consistently to every single document.

6. Edge Cases

Case	Result
A script file WITHOUT execute permission	<code>./script.sh</code> fails with "Permission denied," even if the file's content is perfectly valid
A <code>chmod 777</code> (everyone gets full rwx)	A serious, common security anti-pattern — grants unrestricted access to EVERYONE, including "others," almost never actually appropriate

7. Common Mistakes

- Using `chmod 777` as a "quick fix" for a permission error, instead of understanding and setting the SPECIFIC, appropriately narrow permission actually needed.
- Forgetting a deployment script needs execute permission (`chmod +x`) before it can be run directly.

8. Frequently Asked Interview Questions

- 1 **What three permission categories does every Linux file have?** Owner, Group, Others (everyone else).
- 2 **What do the numbers in `chmod 755` mean?** Each digit represents a permission SET (owner, group, others) as a sum of read=4, write=2, execute=1 — 7 = read+write+execute, 5 = read+execute.
- 3 **Why is `chmod 777` generally considered a bad practice?** It grants unrestricted read/write/execute access to literally EVERYONE on the system, almost never actually the appropriate, intentional access level needed.

TOPIC 5: SSH

1. Introduction

What is it? SSH (Secure Shell) is the ENCRYPTED protocol used to remotely connect to and control a Linux server — this is EXACTLY the mechanism you already used in the AWS section's EC2 topic to connect to your instance and deploy your Spring Boot jar.

2. Recap and Deeper Detail — Key-Based Authentication

```
ssh -i my-key.pem ec2-user@<instance-public-ip> # exactly the command from the AWS section!
```

SSH typically uses **PUBLIC-KEY cryptography** rather than passwords: your `.pem` file is the PRIVATE key (kept secret, NEVER shared), and the SERVER holds the corresponding PUBLIC key — the server can VERIFY you hold the matching private key WITHOUT that private key ever being transmitted over the network, a fundamentally more secure model than password-based login.

3. Generating Your Own SSH Key Pair

```
ssh-keygen -t ed25519 -C "your_email@example.com" # generates a NEW private/public key pair
cat ~/.ssh/id_ed25519.pub # view your PUBLIC key (safe to share/upload)
```

4. Copying Files Over SSH — `scp` (Recap From AWS)

```
scp -i my-key.pem target/app.jar ec2-user@<instance-ip>:/home/ec2-user/
```

5. Real Life Analogy

- **Bank:** SSH's key-based authentication is like a customer proving their identity using a UNIQUE, mathematically-verifiable digital signature that the bank can CONFIRM matches their known public identity, WITHOUT the customer ever needing to hand over or transmit their actual private signing device itself — far more secure than simply reciting a shared password that could be overheard or intercepted.

6. Frequently Asked Interview Questions

- 1 **What is SSH?** An encrypted protocol for securely connecting to and controlling a remote server.
- 2 **Why is key-based SSH authentication considered more secure than password authentication?** The private key never travels over the network at all — the server verifies the connection using public-key cryptography, without the sensitive credential itself ever being transmitted or exposed to interception.
- 3 **What command copies a file to a remote server over SSH?** `scp`.

TOPIC 6: PROCESSES

1. Introduction

What is it? A process is a RUNNING INSTANCE of a program (your `java -jar app.jar` command, once executed, becomes a process) — Linux provides tools to inspect, monitor, and control these running processes.

2. Key Commands

```
ps aux # list ALL currently running processes
ps aux | grep java # find SPECIFICALLY your running Java process (recap: piping, Topic 2!)
top # LIVE, continuously updating view of running processes (CPU/memory usage)
kill 12345 # terminate the process with PID (Process ID) 12345
kill -9 12345 # FORCE kill (SIGKILL) - use only if a graceful 'kill' didn't work
nohup java -jar app.jar & # recap from the AWS section - runs in the BACKGROUND,
# SURVIVING even after you disconnect your SSH session
```

3. Foreground vs Background Processes

```
java -jar app.jar # FOREGROUND - ties up your terminal; closing the SSH session KILLS it
java -jar app.jar & # BACKGROUND - the '&' returns control to your terminal immediately,
# but STILL dies if the SSH session itself closes, UNLESS combined with 'nohup'
```

4. Real Life Analogy

- **Restaurant:** A process is like a SPECIFIC, currently-active cooking task on the stove — `ps aux/top` is like a manager glancing across the ENTIRE kitchen to see everything currently cooking and how much attention/resources each dish is using, and `kill` is like explicitly telling ONE specific dish's preparation to STOP.

5. Frequently Asked Interview Questions

- 1 **What is a process?** A running instance of a program.
- 2 **What does `kill -9` do differently from a plain `kill`?** `kill` sends a polite termination signal the process CAN choose to handle gracefully (cleanup, closing connections); `kill -9` (SIGKILL) forces IMMEDIATE termination, which the process cannot intercept or gracefully respond to at all.
- 3 **Why does a background process (`&`) still die when you close your SSH session, unless combined with `nohup`?** By default, background processes are still tied to the SHELL SESSION that launched them; `nohup` explicitly detaches the process from that session's lifecycle, letting it survive disconnection.

TOPIC 7: CRON JOBS

1. Introduction

What is it? Cron is Linux's built-in TASK SCHEDULER — running commands automatically at specified times/intervals, directly analogous to Spring's `@Scheduled` (Spring Boot section), but operating at the OPERATING SYSTEM level rather than within a specific Java application.

2. Code Example — Editing the Crontab

```
crontab -e # opens YOUR user's crontab file for editing

# Cron syntax: minute hour day-of-month month day-of-week command
0 2 * * * /opt/myapp/backup.sh # run backup.sh EVERY DAY at 2:00 AM
*/15 * * * * curl http://localhost:8080/actuator/health # check health EVERY 15 MINUTES
0 0 1 * * /opt/myapp/monthly-report.sh # run on the 1st day of EVERY MONTH, at midnight
```

3. Cron vs Spring's @Scheduled — When to Use Which

Aspect	Cron	Spring @Scheduled
Scope	System-wide, ANY command/script	Only within a RUNNING Spring Boot application
Runs even if the app is down?	Yes — independent of any specific application	No — requires the application itself to be running
Best for	System maintenance (log rotation, OS-level backups), tasks independent of your app	Application-specific business logic tied to your app's own lifecycle

4. Real Life Analogy

- **Restaurant:** Cron is like a restaurant's building-wide automated systems (the security alarm arming itself every night at closing time, regardless of whether any specific chef is on duty) — as opposed to @Scheduled (Spring Boot), which is more like a SPECIFIC chef's own personal routine that only runs WHILE that chef is actually working their shift.

5. Frequently Asked Interview Questions

- 1 **What is cron?** Linux's built-in task scheduler, running commands automatically at specified times/intervals.
- 2 **How does cron's scope differ from Spring's @Scheduled?** Cron operates at the OS level, independent of any specific application (running even if your app is down); @Scheduled only runs while the Spring Boot application itself is actively running.
- 3 **What does the cron expression */15 * * * * mean?** Run every 15 minutes.

TOPIC 8: SHELL SCRIPTING — BASICS

1. Introduction

What is it? A shell script is a text file containing a SEQUENCE of Linux commands, executed together — automating multi-step tasks (like deploying your Spring Boot app) that would otherwise require manually typing many individual commands.

2. Code Example — A Deployment Script

```
#!/bin/bash # SHEBANG - tells Linux this is a bash script

echo "Starting deployment..."

APP_NAME="employee-api"
JAR_FILE="target/${APP_NAME}-1.0.0.jar"

if [ ! -f "$JAR_FILE" ]; then # IF the jar file does NOT exist...
echo "ERROR: Jar file not found. Run 'mvn package' first."
exit 1 # exit with a NON-ZERO code, signaling FAILURE
fi

echo "Stopping any existing instance..."
pkill -f "$APP_NAME" || true # stop the OLD version (Topic 6!), '|| true' prevents
# the script from failing if nothing was running

echo "Starting new instance..."
nohup java -jar "$JAR_FILE" > app.log 2>&1 & # recap Topic 6's background process technique!

echo "Deployment complete!"

chmod +x deploy.sh # recap Topic 4 - MUST be executable to run directly!
./deploy.sh
```

3. Real Life Analogy

- **Restaurant:** A shell script is like a WRITTEN, standardized closing-procedure checklist a manager follows EVERY night — instead of relying on memory to perform each step individually, EVERY time, in the right order, the ENTIRE sequence is automated into ONE repeatable, reliable procedure.

4. Frequently Asked Interview Questions

- 1 **What is a shell script?** A text file containing a sequence of Linux commands, automating multi-step tasks.
- 2 **What does the "shebang" line (#!/bin/bash) do?** Tells the operating system which interpreter (here, bash) should execute the rest of the script's contents.
- 3 **Why must a shell script be made executable (chmod +x) before running it directly (./script.sh)?**
Recapping Topic 4 — Linux requires explicit execute permission before allowing a file to be run as a program.

TOPIC 9: NGINX

1. Introduction

What is it? Nginx is a widely-used web server and REVERSE PROXY — commonly sitting IN FRONT of your Spring Boot application, handling incoming HTTP requests, SSL/TLS termination, and forwarding traffic to your actual application — conceptually similar to the AWS Load Balancer (AWS section), but running as SOFTWARE on a server you manage yourself.

2. Code Example — A Basic Reverse Proxy Configuration

```
# /etc/etc/nginx/nginx.conf (or a file in /etc/nginx/conf.d/)
server {
    listen 80;
    server_name api.mycompany.com;

    location / {
        proxy_pass http://localhost:8080; # forward ALL traffic to your Spring Boot app,
        proxy_set_header Host $host; # running on port 8080 on the SAME server
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

3. Why Put Nginx IN FRONT of Spring Boot's Embedded Tomcat?

- **SSL/TLS termination:** Nginx handles the HTTPS encryption/decryption, so your Spring Boot app itself only needs to deal with plain HTTP internally.
- **Serving STATIC content** (your React build, from the React section) EFFICIENTLY, without involving your Java application at all for those requests.
- **A single, standard entry point** on port 80/443, while your actual Spring Boot app can run on any internal port.

4. Real Life Analogy

- **Bank:** Nginx is like a bank's FRONT RECEPTION DESK — handling the initial customer greeting/ID verification (SSL/routing) and directing customers to the CORRECT internal department (your application), rather than every single customer walking directly into the actual back-office processing room (your Spring Boot app) themselves.

5. Frequently Asked Interview Questions

- 1 **What is Nginx commonly used for in a Java web deployment?** As a reverse proxy sitting in front of the Spring Boot application, handling incoming HTTP requests, SSL termination, and static file serving before forwarding to the actual application.
- 2 **What does `proxy_pass` do in an Nginx configuration?** Forwards incoming requests to a specified backend address (like your Spring Boot app running on `localhost:8080`).
- 3 **Why handle SSL/TLS termination at Nginx rather than directly in Spring Boot?** It centralizes certificate management and encryption/decryption at one well-understood layer, letting the application itself focus purely on business logic over plain HTTP internally.

TOPIC 10: LOGS

1. Introduction

What is it? This topic ties together everything — where your Spring Boot application's LOGS (recall the Spring Boot Logging topic's SLF4J/Logback output) actually END UP on a real Linux server, and the standard tools for finding, filtering, and monitoring them.

2. Common Log Locations

```
/var/log/nginx/access.log # every request Nginx received
/var/log/nginx/error.log # Nginx-level errors
~/app.log # wherever YOU redirected your Spring Boot app's output (Topic 6's example)
journalctl -u myapp.service # if running as a systemd SERVICE, view its logs this way instead
```

3. Practical Log Investigation (Recapping Topic 2's Commands, Applied Together)

```
tail -f app.log # watch LIVE, as new log lines appear
grep "ERROR" app.log | tail -20 # find the last 20 ERROR lines
grep -c "OutOfMemoryError" app.log # COUNT how many times a specific error occurred
awk '{print $1}' access.log | sort | uniq -c | sort -rn | head # a classic combo: extract IP
addresses,
# count occurrences, sort by frequency -
# find your TOP requesting IPs
```

4. Real Life Analogy

- **Bank:** Log files are like a bank's COMPLETE, timestamped transaction audit trail — `tail -f` is like watching new entries appear on the ledger LIVE as they happen, and `grep/awk` combinations are like a skilled auditor efficiently SEARCHING and SUMMARIZING thousands of entries to answer a SPECIFIC question ("how many failed login attempts happened today?") without manually reading every single line.

5. Frequently Asked Interview Questions

- 1 **Where does Nginx typically store its access and error logs?** `/var/log/nginx/access.log` and `/var/log/nginx/error.log`.
- 2 **How would you view logs for an application running as a systemd service?** `journalctl -u <service-name>`.
- 3 **How would you find the last 20 lines containing "ERROR" in a log file?** `grep "ERROR" app.log | tail -20`.

6. Revision Notes (Entire Linux Section Consolidated)

LINUX — CHEAT SHEET

=====

WHY IT MATTERS: EC2 (AWS), Docker base images, CI/CD runners - your ENTIRE deployment target runs on Linux

FILE SYSTEM: single ROOT (/), /etc (config), /var/log (logs), /home (user dirs)
COMMANDS: ls/cd/cp/mv/rm, cat/less/tail -f (live log watching), grep (search), find, | (pipe - chain commands), > / 2>&1 (redirect output)

USERS & GROUPS: multi-user OS (like AWS IAM, but at the OS level), sudo (temporary elevated privileges - same "least privilege" principle as IAM)

PERMISSIONS: owner/group/others, rwx = 4+2+1, chmod 755/644, chmod +x for scripts, NEVER chmod 777 (unrestricted access to everyone)

SSH: encrypted remote connection, KEY-BASED auth (private key never transmitted) - the EXACT mechanism used to deploy in the AWS EC2 topic

PROCESSES: ps aux / top (inspect), kill / kill -9 (terminate), nohup ... & (survives disconnecting your SSH session)

CRON JOBS: OS-level scheduler (vs Spring's @Scheduled - app-level only, needs the app running) - minute hour day month weekday syntax

SHELL SCRIPTING: #!/bin/bash shebang, chmod +x to run, automates multi-step deployment/maintenance tasks reliably

NGINX: reverse proxy in front of Spring Boot's embedded Tomcat - SSL termination, static file serving, proxy_pass to your app's internal port (conceptually like AWS's Load Balancer, but self-managed software)

LOGS: /var/log/nginx/*, your app's redirected output, journalctl for systemd services - tail -f / grep / awk for live monitoring and investigation

7. Homework (Covers Entire Linux Section)

- 1 SSH into your EC2 instance from the AWS section, and practice navigating with `ls`, `cd`, `pwd`, and viewing your app's log with `tail -f`.
- 2 Create a non-root user, add it to a custom group, and set specific file permissions using `chmod/chown` for a shared application directory.
- 3 Write and run a deployment shell script that checks for the jar file's existence, stops any running instance, and starts a new one in the background with `nohup`.
- 4 Set up a cron job that curls your Spring Boot app's `/actuator/health` endpoint every 5 minutes, logging the result to a file.
- 5 Install and configure Nginx as a reverse proxy in front of your locally-running Spring Boot application.

FINAL SUMMARY — LINUX COMPLETE

This concludes the **Linux** section — 10 topics:

- 1 **File System** — the single-root hierarchy underlying your EC2/Docker/CI environments.

- 2 **Commands** — navigation, piping, redirection.
- 3 **Users & Groups** — the OS-level parallel to AWS IAM.
- 4 **Permissions** — `rx`, `chmod`, `chown`, and why `777` is dangerous.
- 5 **SSH** — key-based authentication, the exact mechanism from the AWS EC2 topic.
- 6 **Processes** — inspecting, killing, and backgrounding your running Java application.
- 7 **Cron Jobs** — OS-level scheduling, contrasted with Spring's `@Scheduled`.
- 8 **Shell Scripting** — automating multi-step deployment tasks.
- 9 **Nginx** — the reverse proxy in front of your Spring Boot app.
- 10 **Logs** — where they live, and how to search/monitor them effectively.

You now understand the actual operating system underlying every server, container, and CI runner used throughout this entire course. Next: System Design, the final section, covering how to architect large-scale systems using everything you've learned.

(END OF LINUX MASTER NOTES — ready to proceed to System Design whenever you say "Next Topic.")